7) Ambulance Routing using A* :

```
import heapq

graph = {
    "S": {"A": 3, "B": 2},
    "A": {"S": 3, "C": 4},
    "B": {"S": 2, "C": 1},
    "C": {"A": 4, "B": 1, "D": 2, "E": 3},
    "D": {"C": 2, "G": 2},
    "E": {"C": 3, "G": 4},
    "G": {"D": 2, "E": 4}
}

h = {
    "S": 6, "A": 6, "B": 4,
    "C": 4, "D": 2, "E": 3, "G": 0
}

def astar(open_road=False):
    roads = {node: edges.copy()
             for node, edges in graph.items()}

    if open_road:
        roads["B"]["D"] = 2
        roads["D"]["B"] = 2

    queue = [(h["S"], 0, "S", ["S"])]
    best = {"S": 0}
    closed = set()

    while queue:
        f, g, node, path = heapq.heappop(queue)

        if node in closed:
            continue

        closed.add(node)
        print("Node:", node, "g:", g,
              "h:", h[node], "f:", f)

        if node == "G":
```

```
            print("Route:", " -> ".join(path))
            print("Total cost:", g)
            return

        for nxt, cost in roads[node].items():
            new_g = g + cost

            if nxt not in closed and new_g < best.get(nxt, float("inf")):
                best[nxt] = new_g
                heapq.heappush(
                    queue,
                    (new_g + h[nxt], new_g, nxt, path + [nxt])
                )

    print("No route found")

print("B-D CLOSED:")
astar(False)

print("\nB-D OPEN:")
astar(True)
```